

JavaScript Objekte

GRG, HOA

Web- and Mobile Computing

12. März 2026

Objekte als Instanzen von Klassen (wie in Java)

- Seit ES6: class-Syntax (ähnlich zu Java)
- Konstruktor mit constructor() (es gibt nur einen Konstruktor)
- Methoden im Klassenrumpf

```
1  class Person {
2      constructor(name, age) {
3          this.name = name;
4          this.age = age;
5      }
6
7      greet() {
8          return "Hi, ␣ich␣bin␣" + this.name;
9      }
10 }
11
12 const p = new Person("Anna", 30);
13 console.log(p.greet());
```

Private Attribute mit # (seit ES2022)

- Mit # definierte Felder sind wirklich privat
- Zugriff nur innerhalb der Klasse möglich
- Kein Zugriff von außen (auch nicht per `this["..."]`)
- Anders als `private` in Java: rein sprachlich erzwungen

```
1  class Person {
2      #age;    // privates Feld (Deklaration erforderlich)
3
4      constructor(name, age) {
5          this.name = name;
6          this.#age = age;
7      }
8
9      getAge() {
10         return this.#age;
11     }
12 }
13
14 const p = new Person("Anna", 30);
15 console.log(p.getAge());    // OK
16 console.log(p.#age);        // Fehler!
```

Vergleich zu Java

- **Ähnlich:**

- Klassen, Konstruktoren, Methoden
- Objekte mit `new`

- **Unterschiede:**

- Keine Typdeklarationen
- Kein echtes Klassen-basiertes Modell, sondern **prototypbasiert**
- Methoden sind Funktionen

```
1 // Java
2 Person p = new Person("Anna", 30);
3
4 // JavaScript
5 const p = new Person("Anna", 30);
```

Vererbung

- extends erzeugt eine Unterklasse
- super(...) ruft den Konstruktor der Oberklasse auf
- super.methode() ruft eine Methode der Oberklasse auf
- super(...) muss im Konstruktor **vor** this stehen!

```
1  class Person {  
2      constructor(name) {  
3          this.name = name;  
4      }  
5  
6      greet() {  
7          return "Hallo_" + this.name;  
8      }  
9  }
```

Vererbung

```
1  class Student extends Person {
2      constructor(name, matrikelNr) {
3          super(name);           // Aufruf Oberklasse
4          this.matrikelNr = matrikelNr;
5      }
6
7      greet() {
8          return super.greet() +
9              "␣(Matrikel:␣" + this.matrikelNr + ")";
10     }
11 }
12
13 const s = new Student("Anna", 12345);
```

Direkte Objektschreibweise (Object Literal)

- Objekte ohne Klasse definierbar
- Schlüssel-Wert-Paare
- Sehr häufige Schreibweise in JS

```
1  const person = {  
2      name: "Anna",  
3      age: 30,  
4      greet: function() {  
5          return "Hi, ich bin " + this.name;  
6      }  
7  };  
8  
9  console.log(person.name);  
10 console.log(person.greet());
```

Dynamische Eigenschaften

- Eigenschaften können hinzugefügt oder entfernt werden
- Anders als in Java!

```
1  const person = { name: "Anna" };  
2  
3  person.age = 30;           // hinzufuegen  
4  delete person.name;       // entfernen  
5  
6  person["city"] = "Berlin"; // alternative Syntax
```


JSON (JavaScript Object Notation)

- Reines Datenformat (keine Funktionen!)
- Wird für Datenaustausch verwendet (z.B. REST)
- Sehr ähnlich zu Object Literals

```
1 // JSON (String)
2 const jsonText = '{"name": "Anna", "age": 30}';
3
4 const obj = JSON.parse(jsonText);
5 const backToJson = JSON.stringify(obj);
```

- JSON: nur Daten
- JS-Objekt: Daten + Methoden

Aufgabe 1 zu Objekten

- 1 Erstellen Sie eine Klasse Konto mit privatem Feld #pin und öffentlichen Feldern username und balance.
Methoden:
 - checkPin(eingabe) → true/false
 - einzahlen(betrag) → wirft Error falls ungültiger Parameter
 - toString() → String-Darstellung des Objektes
 - toPublicJSON() → JSON nur mit öffentlichen Daten (also ohne #pin)
- 2 Erzeugen Sie ein Objekt konto = new Konto('sam2026', 1234) und testen Sie: checkPin(1111) sowie checkPin(1234).
- 3 Testen Sie die einzahlen-Methode. Nur gültige Parameter sollen akzeptiert werden.
- 4 Erstellen Sie eine toString()-Methode, die den Benutzernamen ausgibt, z. B. Benutzer: sam1234. Verwenden Sie dafür Template-Strings:

```
`Variable: ${abc}`
```

Aufgabe 2 zu Objekten

Eine Lernplattform speichert zu jedem Teilnehmer ein Profil.

Startcode:

```
const profil = {  
  username: "sam2026",  
  punkte: 42,  
  aktiv: true,  
  kontakt: {  
    email: "sam@example.com",  
    ort: "Graz"  
  },  
  interessen: ["JavaScript", "Web", "UI"]  
};
```

- 1 Geben Sie username, punkte und die Stadt in einem Satz auf der Konsole aus.

Aufgabe 2 zu Objekten (Fortsetzung)

- 1 Ergänzen Sie das Objekt um `level` mit dem Wert `"Beginner"`. Erhöhen Sie danach `punkte` um 8.
- 2 Entfernen Sie die Eigenschaft `aktiv` mit `delete`. Prüfen Sie anschließend mit `console.log`, ob sie noch existiert.
- 3 Wandeln Sie `profil` in JSON um (`JSON.stringify`) und speichern Sie das Ergebnis in `profilJson`. Konvertieren Sie es danach mit `JSON.parse` zurück in `profilKopie`.
- 4 Fügen Sie `profilKopie` eine neue Eigenschaft `letzterLogin` hinzu (z. B. `"2026-03-11"`).